

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("abdulhasibuddin/plant-doc-dataset")

print("Path to dataset files:", path)
```

[Show hidden output](#)

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("emmarex/plantdisease")

print("Path to dataset files:", path)
```

[Show hidden output](#)

```
import os

plantdoc_path = "/root/.cache/kagglehub/datasets/abdulhasibuddin/plant-doc-dataset/versions/1"
plantvillage_path = "/kaggle/input/plantdisease"

print("PLANTDOC")
print(os.listdir(plantdoc_path))

print("\nPLANTVILLAGE")
print(os.listdir(plantvillage_path))
```

[Show hidden output](#)

```
import os

plantdoc_path = "/root/.cache/kagglehub/datasets/abdulhasibuddin/plant-doc-dataset/versions/1/PlantDoc-Dataset"
plantvillage_path = "/kaggle/input/plantdisease"

print("=== PLANTDOC ===")
print(os.listdir(plantdoc_path))
```

```
print("\n=== PLANTVILLAGE ===")
for folder in os.listdir(plantvillage_path):
    print(folder)
```

[Show hidden output](#)

```
import os

plantvillage_path = "/kaggle/input/plantdisease"

for folder in ["PlantVillage", "plantvillage"]:
    path = os.path.join(plantvillage_path, folder)

    print(f"\n=== {folder} ===")
    items = os.listdir(path)

    print("Number of items:", len(items))
    print("First 15 items:")
    print(items[:15])
```

[Show hidden output](#)

```
import os

plantdoc_train = "/root/.cache/kagglehub/datasets/abdulhasibuddin/plant-doc-dataset/versions/1/PlantDoc-Dataset/train"
plantdoc_test = "/root/.cache/kagglehub/datasets/abdulhasibuddin/plant-doc-dataset/versions/1/PlantDoc-Dataset/test"

print("=== PLANTDOC TRAIN ===")
print(os.listdir(plantdoc_train))

print("\n=== PLANTDOC TEST ===")
print(os.listdir(plantdoc_test))
```

[Show hidden output](#)

```
import os

plantdoc_train = "/root/.cache/kagglehub/datasets/abdulhasibuddin/plant-doc-dataset/versions/1/PlantDoc-Dataset/train"
plantdoc_test = "/root/.cache/kagglehub/datasets/abdulhasibuddin/plant-doc-dataset/versions/1/PlantDoc-Dataset/test"
```

```

plantvillage = "/kaggle/input/plantdisease/PlantVillage"

print("=== PLANTDOC TRAIN COUNTS ===")

for cls in sorted(os.listdir(plantdoc_train)):
    path = os.path.join(plantdoc_train, cls)
    if os.path.isdir(path):
        print(f"{cls}: {len(os.listdir(path))}")

print("\n=== PLANTVILLAGE COUNTS ===")

for cls in sorted(os.listdir(plantvillage)):
    path = os.path.join(plantvillage, cls)
    if os.path.isdir(path):
        print(f"{cls}: {len(os.listdir(path))}")

```

[Show hidden output](#)

```

import os
import shutil

# Dataset locations
plantdoc_train = "/root/.cache/kagglehub/datasets/abdulhasibuddin/plant-doc-dataset/versions/1/PlantDoc-Dataset/train"
plantvillage = "/kaggle/input/plantdisease/PlantVillage"

# Where we will create the combined dataset
output_path = "/content/florabot_dataset"

# Create output folder
os.makedirs(output_path, exist_ok=True)

# Mapping: final class name -> PlantVillage folder + PlantDoc folder
class_mapping = {
    "Pepper_Bacterial_Spot": (
        "Pepper__bell__Bacterial_spot",
        "Bell_pepper leaf spot"
    ),
    "Potato_Early_Blight": (
        "Potato__Early_blight",
        "Potato leaf early blight"
    )
}

```

```

    ),
    "Potato_Late_Blight": (
        "Potato__Late_blight",
        "Potato leaf late blight"
    ),
    "Tomato_Bacterial_Spot": (
        "Tomato_Bacterial_spot",
        "Tomato leaf bacterial spot"
    ),
    "Tomato_Early_Blight": (
        "Tomato_Early_blight",
        "Tomato Early blight leaf"
    ),
    "Tomato_Late_Blight": (
        "Tomato_Late_blight",
        "Tomato leaf late blight"
    ),
    "Tomato_Leaf_Mold": (
        "Tomato_Leaf_Mold",
        "Tomato mold leaf"
    ),
    "Tomato_Septoria": (
        "Tomato_Septoria_leaf_spot",
        "Tomato Septoria leaf spot"
    ),
    "Tomato_Yellow_Leaf_Curl_Virus": (
        "Tomato__Tomato_YellowLeaf__Curl_Virus",
        "Tomato leaf yellow virus"
    ),
    "Tomato_Mosaic_Virus": (
        "Tomato__Tomato_mosaic_virus",
        "Tomato leaf mosaic virus"
    ),
}

```

```

# Copy images from both datasets
for final_class, (pv_class, pd_class) in class_mapping.items():

```

```

    class_output = os.path.join(output_path, final_class)
    os.makedirs(class_output, exist_ok=True)

```

```

# PlantVillage images

```

```
pv_path = os.path.join(plantvillage, pv_class)

for file in os.listdir(pv_path):
    src = os.path.join(pv_path, file)
    dst = os.path.join(class_output, "PV_" + file)
    shutil.copy2(src, dst)

# PlantDoc images
pd_path = os.path.join(plantdoc_train, pd_class)

for file in os.listdir(pd_path):
    src = os.path.join(pd_path, file)
    dst = os.path.join(class_output, "PD_" + file)
    shutil.copy2(src, dst)

print("Dataset created successfully!")
print("\nLocation:", output_path)

print("\nClasses:")
for cls in sorted(os.listdir(output_path)):
    count = len(os.listdir(os.path.join(output_path, cls)))
    print(f"{cls}: {count} images")
```

[Show hidden output](#)

```
import os
import shutil
from sklearn.model_selection import train_test_split

source = "/content/florabot_dataset"
destination = "/content/florabot_split"

classes = os.listdir(source)

for split in ["train", "val", "test"]:
    for cls in classes:
        os.makedirs(os.path.join(destination, split, cls), exist_ok=True)

for cls in classes:
    class_path = os.path.join(source, cls)
    images = [
```

```
        f for f in os.listdir(class_path)
        if f.lower().endswith((".jpg", ".jpeg", ".png"))
    ]

train_files, temp_files = train_test_split(
    images,
    test_size=0.30,
    random_state=42
)

val_files, test_files = train_test_split(
    temp_files,
    test_size=0.50,
    random_state=42
)

for f in train_files:
    shutil.copy2(
        os.path.join(class_path, f),
        os.path.join(destination, "train", cls, f)
    )

for f in val_files:
    shutil.copy2(
        os.path.join(class_path, f),
        os.path.join(destination, "val", cls, f)
    )

for f in test_files:
    shutil.copy2(
        os.path.join(class_path, f),
        os.path.join(destination, "test", cls, f)
    )

print("Dataset split completed!")
print("Location:", destination)
```

[Show hidden output](#)

```
import os
```

```
base = "/content/florabot_split"

for split in ["train", "val", "test"]:
    total = 0
    print(f"\n=== {split.upper()} ===")

    for cls in sorted(os.listdir(os.path.join(base, split))):
        count = len(os.listdir(os.path.join(base, split, cls)))
        total += count
        print(f"{cls}: {count}")

    print("TOTAL:", total)
```

```
=== TRAIN ===
Pepper_Bacterial_Spot: 741
Potato_Early_Blight: 776
Potato_Late_Blight: 767
Tomato_Bacterial_Spot: 1557
Tomato_Early_Blight: 751
Tomato_Late_Blight: 1407
Tomato_Leaf_Mold: 725
Tomato_Mosaic_Virus: 291
Tomato_Septoria: 1335
Tomato_Yellow_Leaf_Curl_Virus: 2293
TOTAL: 10643
```

```
=== VAL ===
Pepper_Bacterial_Spot: 159
Potato_Early_Blight: 166
Potato_Late_Blight: 165
Tomato_Bacterial_Spot: 334
Tomato_Early_Blight: 161
Tomato_Late_Blight: 301
Tomato_Leaf_Mold: 156
Tomato_Mosaic_Virus: 63
Tomato_Septoria: 286
Tomato_Yellow_Leaf_Curl_Virus: 492
TOTAL: 2283
```

```
=== TEST ===
Pepper_Bacterial_Spot: 159
Potato_Early_Blight: 167
Potato_Late_Blight: 165
```

```
Tomato_Bacterial_Spot: 334
Tomato_Early_Blight: 162
Tomato_Late_Blight: 302
Tomato_Leaf_Mold: 156
Tomato_Mosaic_Virus: 63
Tomato_Septoria: 287
Tomato_Yellow_Leaf_Curl_Virus: 492
TOTAL: 2287
```

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.applications import MobileNetV2

# =====
# 1. Dataset paths
# =====

train_dir = "/content/florabot_split/train"
val_dir = "/content/florabot_split/val"

# =====
# 2. Settings
# =====

IMG_SIZE = (224, 224)
BATCH_SIZE = 32

# =====
# 3. Load training dataset
# =====

train_data = tf.keras.utils.image_dataset_from_directory(
    train_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="categorical",
    shuffle=True,
    seed=42
)

# =====
# 4. Load validation dataset
```

```
# =====

val_data = tf.keras.utils.image_dataset_from_directory(
    val_dir,
    image_size=IMG_SIZE,
    batch_size=BATCH_SIZE,
    label_mode="categorical",
    shuffle=False
)

# =====
# 5. Get class names
# =====

class_names = train_data.class_names
num_classes = len(class_names)

print("\nClasses:")
for i, name in enumerate(class_names):
    print(i, "->", name)

print("\nNumber of classes:", num_classes)

# =====
# 6. Improve dataset loading
# =====

AUTOTUNE = tf.data.AUTOTUNE

train_data = train_data.prefetch(AUTOTUNE)
val_data = val_data.prefetch(AUTOTUNE)

# =====
# 7. Load MobileNetV2
# =====

base_model = MobileNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(224, 224, 3)
)
```

```
# Freeze pretrained CNN layers
base_model.trainable = False

# =====
# 8. Build our CNN classifier
# =====

model = models.Sequential([
    base_model,

    layers.GlobalAveragePooling2D(),

    layers.Dropout(0.2),

    layers.Dense(
        num_classes,
        activation="softmax"
    )
])

# =====
# 9. Compile model
# =====

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

# =====
# 10. Display model
# =====

model.summary()
```

[Show hidden output](#)

```
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint, ReduceLROnPlateau

# Callbacks
```

```
early_stopping = EarlyStopping(
    monitor="val_loss",
    patience=3,
    restore_best_weights=True,
    verbose=1
)

checkpoint = ModelCheckpoint(
    "/content/florabot_mobilenetv2_best.keras",
    monitor="val_accuracy",
    save_best_only=True,
    verbose=1
)

reduce_lr = ReduceLROnPlateau(
    monitor="val_loss",
    factor=0.2,
    patience=2,
    min_lr=1e-6,
    verbose=1
)

# Train the model
history = model.fit(
    train_data,
    validation_data=val_data,
    epochs=15,
    callbacks=[
        early_stopping,
        checkpoint,
        reduce_lr
    ]
)
```

[Show hidden output](#)

```
from google.colab import drive

drive.mount('/content/drive')
```

[Show hidden output](#)

```
import tensorflow as tf

test_dir = "/content/florabot_split/test"

test_ds = tf.keras.utils.image_dataset_from_directory(
    test_dir,
    image_size=(224, 224),
    batch_size=32,
    label_mode="categorical",
    shuffle=False
)

print("Test dataset created.")
print(test_ds.class_names)
```

[Show hidden output](#)

```
test_loss, test_accuracy = model.evaluate(test_ds)

print(f"Test accuracy: {test_accuracy * 100:.2f}%")
print(f"Test loss: {test_loss:.4f}")
```

[Show hidden output](#)

```
import numpy as np
from sklearn.metrics import classification_report

y_true = np.concatenate([y.numpy() for _, y in test_ds], axis=0)
y_true = np.argmax(y_true, axis=1)

y_prob = model.predict(test_ds)
y_pred = np.argmax(y_prob, axis=1)

print(classification_report(
    y_true,
    y_pred,
    target_names=test_ds.class_names,
    digits=3
))
```

[Show hidden output](#)

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_true, y_pred)

plt.figure(figsize=(12, 9))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
    xticklabels=test_ds.class_names,
    yticklabels=test_ds.class_names
)

plt.xlabel("Predicted label")
plt.ylabel("True label")
plt.title("Flora-Bot Confusion Matrix")
plt.xticks(rotation=45, ha="right")
plt.yticks(rotation=0)
plt.tight_layout()
plt.show()
```

[Show hidden output](#)

```
import tensorflow as tf

# Unfreeze only the top 30 layers of MobileNetV2
base_model.trainable = True

for layer in base_model.layers[:-30]:
    layer.trainable = False

# Recompile with a very small learning rate
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1e-5),
    loss="categorical_crossentropy",
```

```
        metrics=["accuracy"]
    )

    print("Fine-tuning prepared.")
    print("Trainable layers:", sum(layer.trainable for layer in base_model.layers))
```

[Show hidden output](#)

```
from tensorflow.keras.callbacks import ModelCheckpoint, EarlyStopping

checkpoint_path = "/content/drive/MyDrive/FloraBot/florabot_mobilenetv2_finetuned_best.keras"

callbacks = [
    ModelCheckpoint(
        checkpoint_path,
        monitor="val_accuracy",
        save_best_only=True,
        mode="max",
        verbose=1
    ),
    EarlyStopping(
        monitor="val_accuracy",
        patience=2,
        restore_best_weights=True,
        mode="max"
    )
]

history_finetune = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=5,
    callbacks=callbacks
)
```

[Show hidden output](#)

```
import tensorflow as tf

train_ds = tf.keras.utils.image_dataset_from_directory(
    "/content/florabot_split/train",
```

```
        image_size=(224, 224),
        batch_size=32,
        label_mode="categorical",
        shuffle=True
    )

    val_ds = tf.keras.utils.image_dataset_from_directory(
        "/content/florabot_split/val",
        image_size=(224, 224),
        batch_size=32,
        label_mode="categorical",
        shuffle=False
    )

    print("Train and validation datasets recreated.")
```

[Show hidden output](#)

```
test_loss_ft, test_accuracy_ft = model.evaluate(test_ds)

print(f"Fine-tuned test accuracy: {test_accuracy_ft * 100:.2f}%")
print(f"Fine-tuned test loss: {test_loss_ft:.4f}")
```

[Show hidden output](#)

```
from tensorflow.keras.models import load_model

model = load_model("/content/florabot_mobilenetv2_best.keras")

print("Original best MobileNetV2 model restored.")
```

[Show hidden output](#)

```
from sklearn.metrics import confusion_matrix
import numpy as np

y_true = np.concatenate([y.numpy() for _, y in test_ds], axis=0)
y_true = np.argmax(y_true, axis=1)

y_prob = model.predict(test_ds)
```

```
y_pred = np.argmax(y_prob, axis=1)

cm = confusion_matrix(y_true, y_pred)

print("Confusion Matrix:")
print(cm)
```

[Show hidden output](#)

```
data_augmentation = tf.keras.Sequential([
    tf.keras.layers.RandomFlip("horizontal"),
    tf.keras.layers.RandomRotation(0.15),
    tf.keras.layers.RandomZoom(0.15),
    tf.keras.layers.RandomContrast(0.15),
])

print("Data augmentation ready.")
```

[Show hidden output](#)

```
# Add augmentation before MobileNetV2
inputs = tf.keras.Input(shape=(224, 224, 3))

x = data_augmentation(inputs)
x = tf.keras.applications.mobilenet_v2.preprocess_input(x)

x = base_model(x, training=False)
x = tf.keras.layers.GlobalAveragePooling2D()(x)
x = tf.keras.layers.Dropout(0.2)(x)
outputs = tf.keras.layers.Dense(10, activation="softmax")(x)

augmented_model = tf.keras.Model(inputs, outputs)

# Keep MobileNetV2 frozen for this experiment
base_model.trainable = False

augmented_model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)
```

```
print("Augmented MobileNetV2 model ready.")
```

[Show hidden output](#)

```
history_augmented = augmented_model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=5
)
```

[Show hidden output](#)

```
test_loss_aug, test_accuracy_aug = augmented_model.evaluate(test_ds)

print(f"Augmented model test accuracy: {test_accuracy_aug * 100:.2f}%")
print(f"Augmented model test loss: {test_loss_aug:.4f}")
```

[Show hidden output](#)

```
augmented_model.save("/content/drive/MyDrive/FloraBot/florabot_mobilenetv2_augmented_best.keras")

print("Augmented model saved successfully.")
```

[Show hidden output](#)

```
import os

print(os.listdir('/content'))
```

[Show hidden output](#)

```
from tensorflow.keras.models import load_model

model = load_model("/content/florabot_mobilenetv2_best.keras")

print("Model loaded successfully!")
```

[Show hidden output](#)

```
print(class_names)
```

[Show hidden output](#)

```
from google.colab import drive
drive.mount('/content/drive')
```

```
from google.colab import files
from tensorflow.keras.utils import load_img
import numpy as np

uploaded = files.upload()

image_path = list(uploaded.keys())[0]

img = load_img(image_path, target_size=(224, 224))
img_array = np.array(img)
img_array = np.expand_dims(img_array, axis=0)

prediction = model.predict(img_array)

predicted_index = np.argmax(prediction[0])
predicted_class = class_names[predicted_index]
confidence = prediction[0][predicted_index] * 100

print("\nImage:", image_path)
print("Prediction:", predicted_class)
print("Confidence:", round(confidence, 2), "%")
```

[Show hidden output](#)

Start coding or [generate](#) with AI.

